NigeriaCompliance Workflow Documentation

Overview

The NigeriaCompliance workflow is a comprehensive financial compliance analysis system designed to process uploaded documents, extract financial data using AI, perform compliance checks, and generate detailed reports. The system is deployed as a FastAPI backend on Railway with a Vercel frontend interface.

Date Created: May 9, 2026
Version: 1.0
Deployment: Railway (Backend) + Vercel (Frontend)

Table of Contents

System Architecture

Backend Components

- **FastAPI Server** (`api/server.py`): REST API endpoints
- **Workflow Engine** (`workflow/`): Core processing logic
- **Document Processing**: PDF, DOCX, XLSX, CSV, TXT, PNG/JPG extraction
- **LLM Integration**: OpenAI GPT models for data interpretation
- **Report Generation**: HTML and PDF report creation

Frontend Components

- **Vercel App**: React/Next.js interface
- **API Client** (`vercel_api_client.js`): JavaScript helpers for API calls
- **File Upload**: Drag-and-drop document upload interface

Infrastructure

- **Railway**: Cloud hosting with automatic deployment from GitHub
- **Docker**: Containerized deployment with system dependencies
- **GitHub**: Version control and CI/CD trigger

API Endpoints

Core Endpoints

`GET /`

Purpose: API status and endpoint listing
Response: JSON with available endpoints and usage notes
Implementation: Returns static endpoint documentation

`GET /health`

Purpose: Health check for load balancers and monitoring
Response: `{"status": "ok"}`
Implementation: Simple status check

`POST /upload`

Purpose: Upload documents to repository
Parameters:

- `file`: File upload (required)
- `department`: Department name (form field, default: "other") **Response:** Upload confirmation with stored path and filename
  Implementation: Saves files to `repository/{department}/` with UUID prefix

`POST /process`

Purpose: Trigger background document processing
Response: Processing status with background job info
Implementation: Starts threaded workflow execution, returns immediately

`GET /process/status`

Purpose: Check processing job status
Response: Current processing state, errors, timestamps
Implementation: Returns PROCESS_STATUS global state

`GET /aggregated`

Purpose: Retrieve processed results
Response: Aggregated financial data JSON
Implementation: Serves `output/aggregated_data.json`

`GET /artifact/{filename}`

Purpose: Download generated reports
Parameters: filename (path parameter)
Response: File download (HTML/PDF reports)
Implementation: Serves files from `output/` directory

Debug Endpoints

`GET /debug/files`

Purpose: List all files in repository
Response: Files organized by department with metadata
Implementation: Scans `repository/` directory structure

`GET /debug/test-extraction`

Purpose: Test document extraction on first file
Response: Extraction results or error details
Implementation: Runs extraction pipeline on one file

`GET /debug/env`

Purpose: Show runtime environment variables
Response: API key presence and configuration values
Implementation: Returns sanitized environment state

`GET /debug/output`

Purpose: List generated output files
Response: Files in output directory with metadata
Implementation: Scans `output/` directory

`GET /debug/processing-files`

Purpose: Show files that will be processed
Response: All files with processed/unprocessed status
Implementation: Compares against `.processed_files.json`

File Processing Pipeline

1. File Discovery (`workflow/ingestion.py`)

Purpose: Locate all documents in the repository for processing

Code Location: `workflow/ingestion.py:discover_files()`

Process:

1. Scans four department directories: `finance/`, `hr/`, `operations/`, `procurement/`
2. Returns list of file dictionaries with department and path information
3. Only processes files in these specific directories

Output: List of `{"department": "Finance", "path": Path(...)}` dictionaries

2. Document Extraction (`workflow/extraction.py`)

Purpose: Extract raw text and tabular data from various file formats

Supported Formats:

- **PDF**: Uses `pdfplumber` for text/tables, `pytesseract` for OCR
- **DOCX**: Uses `python-docx` for text and tables
- **XLSX**: Uses `pandas` for spreadsheet data
- **CSV**: Uses `pandas` for tabular data
- **TXT**: Direct text reading
- **Images**: OCR using `pytesseract`

Process:

1. Detect file type by extension
2. Call appropriate extraction function
3. Return standardized format: `{"raw_text": str, "raw_tables": list, "source_path": str, "file_type": str}`

LLM Integration: None at this stage - raw data extraction only

3. Data Interpretation (`workflow/genai_agents.py:interpretation_agent()`)

Purpose: Use AI to classify documents and extract structured financial data

LLM Prompt:

```
You are a senior financial analyst and document classifier.
You receive JSON with raw_text and raw_tables from a financial document.
Your tasks:
1) Identify the most likely department (Finance, HR, Procurement, Operations, or Other).
2) Identify the reporting period (e.g., 'Q1 2025', 'FY 2024') if present.
3) Extract key financial metrics as a flat object (e.g., revenue, net_profit, total_payroll, total_vendor_spend).
4) Extract important notes as an array of strings.
5) List missing fields that should be present but aren't found.
6) Provide a confidence score (0-1) for the extraction quality.

Respond ONLY with valid JSON matching this structure:
{
  "department": "Finance",
  "period": "Q1 2025",
  "metrics": {"revenue": 1000000, "net_profit": 250000},
  "notes": ["Strong quarterly performance", "Increased operational costs"],
  "missing_fields": ["total_payroll", "vendor_spend_breakdown"],
  "confidence": 0.85
}
```

Process:

1. Send raw document data to OpenAI GPT model
2. Parse JSON response for structured data
3. Validate response format and handle errors
4. Return interpreted financial data

Fallback: If LLM unavailable, returns stub data with “LLM unavailable” notes

4. Data Aggregation (`workflow/aggregation.py`)

Purpose: Combine data from multiple documents into department-wise summaries

Process:

1. Group records by department
2. Aggregate metrics across documents
3. Calculate totals and averages
4. Preserve period information

Output: Hierarchical data structure with departmental breakdowns

5. Compliance Checking (`workflow/compliance.py`)

Purpose: Apply Nigerian financial compliance rules to aggregated data

Rules Checked:

- Revenue thresholds and reporting requirements
- Payroll compliance and tax obligations
- Vendor spend limits and procurement rules
- Operational cost controls
- Period reporting completeness

Process:

1. Evaluate aggregated data against compliance thresholds
2. Generate pass/fail status
3. Identify specific issues and violations
4. Calculate compliance scores

Output: `{"status": "PASS/FAIL/PARTIAL", "issues": [...], "score": 0.95}`

6. Risk Analysis (`workflow/genai_agents.py:risk_analysis_agent()`)

Purpose: Generate executive-level risk assessment using AI

LLM Prompt:

```
You are a senior compliance officer conducting a risk analysis for Nigerian financial compliance.

Based on the following compliance check results, provide a comprehensive risk assessment:

Detected issues:
{issues_text}

Provide a detailed risk analysis covering:
- Overall compliance risk level (Low/Medium/High/Critical)
- Key risk areas identified
- Potential regulatory implications
- Recommended mitigation actions
- Timeline for resolution

Focus on Nigerian financial regulations and compliance requirements.
```

Process:

1. Format compliance issues for LLM consumption
2. Generate risk narrative using OpenAI
3. Return detailed risk assessment

Fallback: Generic risk analysis message if LLM unavailable

7. Report Generation (`workflow/reporting.py`)

Purpose: Create HTML and PDF reports with charts and executive summaries

Components:

- **Charts**: Revenue, costs, compliance scores using matplotlib
- **HTML Report**: Interactive web report with embedded charts
- **PDF Report**: Print-ready compliance report
- **Executive Summary**: AI-generated narrative using report_writer_agent

Report Writer Agent Prompt:

```
You are a financial reporting specialist preparing an executive summary for Nigerian compliance.

Based on the aggregated financial data and compliance status, create a comprehensive executive summary:

Financial Data:
{aggregated_data}

Compliance Status: {status}
Risk Assessment: {risk_narrative}

Create an executive summary covering:
- Overall financial performance highlights
- Compliance status and key findings
- Risk areas and mitigation recommendations
- Future outlook and recommendations

Keep the summary professional, concise, and focused on key stakeholders' concerns.
```

Process:

1. Generate matplotlib charts for key metrics
2. Create HTML report with embedded charts and data tables
3. Generate PDF version using reportlab
4. Save all artifacts to output directory

LLM Integration

Primary LLM: OpenAI GPT-4o-mini

Configuration:

- Model: `gpt-4o-mini` (configurable via `OPENAI_MODEL` env var)
- Temperature: 0.2 (consistent, factual responses)
- API Key: Required via `OPENAI_API_KEY` environment variable
- Timeout: 60 seconds per call
- Retries: 3 attempts with exponential backoff

Fallback System

Priority Order:

1. OpenAI API (if key present)
2. Local Ollama (if available)
3. Stub responses (if both unavailable)

Stub Responses:

- Interpretation: Returns “LLM unavailable: used fallback extraction”
- Risk Analysis: Generic compliance review message
- Report Summary: Draft status notification

Error Handling

OpenAI Errors:

- Network timeouts
- API rate limits
- Invalid API keys
- Model unavailability

Recovery:

- Automatic retry with backoff
- Graceful fallback to stubs
- Detailed error logging

Workflow Sections

Ingestion Module (`workflow/ingestion.py`)

Purpose: File discovery and initial processing setup

Functions:

- `discover_files(repo_dir)` : Scan department directories for documents

Key Features:

- Department-based organization (finance, hr, operations, procurement)
- File type agnostic (supports multiple formats)
- Incremental processing (tracks processed files)

Extraction Module (`workflow/extraction.py`)

Purpose: Raw data extraction from documents

Functions:

- `extract_record(file_info)` : Main extraction dispatcher
- Format-specific extractors: `extract_pdf()` , `extract_word()` , `extract_excel()` , etc.

Dependencies:

- `pdfplumber`: PDF text and table extraction
- `pytesseract`: OCR for images and scanned PDFs
- `python-docx`: Word document processing
- `pandas`: Spreadsheet and CSV handling
- `Pillow`: Image processing

GenAI Agents (`workflow/genai_agents.py`)

Purpose: AI-powered data interpretation and analysis

Agents:

1. **Interpretation Agent**: Document classification and metric extraction
2. **Risk Analysis Agent**: Compliance risk assessment
3. **Report Writer Agent**: Executive summary generation

Features:

- OpenAI integration with fallback system
- Structured JSON responses
- Error handling and retry logic
- Confidence scoring

Aggregation Module (`workflow/aggregation.py`)

Purpose: Combine and summarize data across documents

Functions:

- `aggregate_records(records)`: Main aggregation logic

Process:

- Department-wise grouping
- Metric summation and averaging
- Period tracking
- Data validation

Compliance Module (`workflow/compliance.py`)

Purpose: Apply Nigerian financial compliance rules

Functions:

- `run_compliance_checks(aggregated_data)`: Rule evaluation

Rules Implemented:

- Revenue reporting thresholds
- Payroll compliance requirements
- Procurement spending limits
- Operational cost controls
- Documentation completeness

Reporting Module (`workflow/reporting.py`)

Purpose: Generate visual reports and summaries

Functions:

- `create_summary_charts()` : Matplotlib chart generation
- `generate_html_report()` : Interactive HTML reports
- `generate_pdf_report()` : Print-ready PDF reports

Features:

- Embedded charts and data visualization
- Professional formatting
- Downloadable artifacts
- Responsive design

Data Flow

Upload Phase

1. User uploads file via `POST /upload`
2. File saved to `repository/{department}/{uuid}_{filename}`
3. Upload confirmation returned

Processing Phase

1. User calls `POST /process`
2. Background thread starts workflow
3. Status tracked in global PROCESS_STATUS

Workflow Execution

1. **Discovery**: Find all unprocessed files in repository
2. **Extraction**: Extract raw text/tables from each file
3. **Interpretation**: LLM classifies and extracts structured data
4. **Aggregation**: Combine data across documents
5. **Compliance**: Apply Nigerian financial rules
6. **Risk Analysis**: Generate risk assessment
7. **Reporting**: Create charts and reports

Retrieval Phase

1. User polls `GET /process/status` for completion
2. User fetches `GET /aggregated` for results
3. User downloads `GET /artifact/{filename}` for reports

Error Handling

API Level Errors

- **404 Not Found**: Missing files or endpoints
- **500 Internal Server Error**: Processing failures
- **Structured Error Responses**: Include error type, traceback, and hints

Processing Errors

- **File Processing**: Unsupported formats, corrupted files
- **LLM Errors**: API failures, timeouts, invalid responses
- **Data Errors**: Missing fields, invalid formats

Recovery Mechanisms

- **Retry Logic**: Exponential backoff for LLM calls
- **Fallbacks**: Stub responses when AI unavailable
- **Graceful Degradation**: Partial results when some files fail

Logging

- **Railway Logs**: Real-time processing status
- **Debug Endpoints**: Detailed error inspection
- **Status Tracking**: Background job monitoring

Deployment Configuration

Environment Variables

Required:

- `OPENAI_API_KEY`: OpenAI API key for LLM functionality

Optional:

- `OPENAI_MODEL`: Model name (default: gpt-4o-mini)
- `FRONTEND_ORIGINS`: CORS allowed origins (default: *)
- `BASE_URL`: Base URL for artifact links
- `LLM_MAX_RETRIES`: Retry attempts (default: 3)
- `LLM_CALL_TIMEOUT`: API timeout seconds (default: 60)

Docker Configuration

Base Image: `python:3.11-slim`

System Dependencies:

- `tesseract-ocr`: OCR functionality
- `poppler-utils`: PDF processing

Build Process:

1. Install system packages
2. Install Python dependencies
3. Copy application code
4. Set CMD for uvicorn server

Railway Deployment

Build Trigger: GitHub push to main branch

Port Configuration: Dynamic port via `${PORT:-8080}`

Health Checks: Automatic via Railway monitoring

Vercel Frontend

Environment Variables:

- `NEXT_PUBLIC_API_BASE_URL`: Railway service URL

API Client: `vercel_api_client.js` with helper functions

Usage Examples

Basic Workflow

```
// Upload document
const upload = await uploadFile(file, 'finance');

// Start processing
const process = await triggerProcess();

// Wait for completion
const status = await triggerProcessAndWait();

// Get results
const results = await getAggregated();
```

Debug Workflow

```
// Check environment
const env = await fetch('/debug/env');

// List files
const files = await fetch('/debug/files');

// Test extraction
const test = await fetch('/debug/test-extraction');
```

Maintenance and Troubleshooting

Common Issues

"Aggregated data not found"

- Processing hasn't completed yet
- Use `/process/status` to check progress
- Wait for `running: false` before fetching results

"No new input files found"

- All files have been processed already
- Upload new documents to trigger processing
- Check `/debug/processing-files` for file status

LLM Errors

- Check `OPENAI_API_KEY` in Railway variables
- Verify API key has sufficient credits
- Check Railway logs for detailed error messages

Monitoring

Railway Dashboard:

- Real-time logs
- Deployment status
- Resource usage

Debug Endpoints:

- `/debug/env`: Environment configuration
- `/debug/files`: Repository contents
- `/debug/output`: Generated files
- `/debug/processing-files`: Processing queue

Future Enhancements

Planned Features

- Multi-tenant support
- Advanced compliance rule engine
- Real-time processing status WebSocket
- Batch processing for multiple files
- Custom compliance rule configuration
- Integration with external financial systems

Performance Optimizations

- Caching for repeated LLM calls
- Parallel file processing
- Database storage for historical data
- CDN for report artifacts

This document provides comprehensive documentation of the NigeriaCompliance workflow system. For technical support or feature requests, refer to the GitHub repository or Railway deployment logs.